

Image-Based Defect Detection for Manufacturing Inspection(Team 10)

This live script will demonstrate the workings of an Image-Based Defect Detection for a manufacturing system developed by a team of transfer college students. In general, a user sends an image from a predetermined dataset through two systems that do the following:

- Classification unit that classifies an image based on:
 - Object type
 - Condition (defect or no defect)
 - Defect type (if applicable)
- Image processing unit that:
 - Overlays an evidence mask for images classified as defective
 - Provides metrics to quantify the area(s) that the defect(s) were detected

These two systems are combined and work together to take the input image and output reliable information that helps speed up manufacturing processes. The live script will showcase

- How each separate system was developed
- The methodology used for development
- How the systems function
- Difficulties and trade-offs during development
- How all systems work together
- Testing of the system for evaluation and robustness
- Interpretation of the whole system evaluation and its purpose
- Summary

Each of these will be spoken about in the relevant sections

Tip before moving through the script: Ensure you clear the output window after every section. That way, outputs have the entire space to display the large amount of output they will have to output down the line.

Table of Contents

Break Down the Problem and Split the Load.....	2
Task 1: Explore and Organize Image Dataset	4
Task 2: Build a Single-Image Inspection Function.....	6
Overview.....	6
Molly.....	6
inspectHazelnut.m Function.....	6
inspectBottle.m Function.....	7
inspectWood.m Function.....	7

Basil.....	8
inspectGrid.m function.....	8
Issues and challenges:.....	9
Results and a brief reflection:.....	10
Task 3: Train and Integrate an AI Classifier into your Single-Image Inspection Function.....	17
Background.....	17
The AI Classifier.....	18
Struggles.....	18
Task 3 Function: Testing the Deliverable function.....	26
Task 3 Checkpoint: Combine outputs into a hybrid inspection result.....	28
How the system functions.....	28
Difficulties and trade-offs.....	29
Task 4: Evaluate Inspection System Performance.....	33
Background.....	33
The Testing Script (runInspectionSuite).....	34
Results from pre-trained network.....	36
Struggles.....	37
Task 5: Test and Evaluate System Robustness.....	38
Background.....	38
Results from pre-trained networks.....	39
System conclusion/summary.....	40
Interpretation of Results.....	41
Summarize your findings by interpreting their physical/engineering meaning:.....	41
What are some limitations of your work?.....	41
What are practical next steps?.....	42

Break Down the Problem and Split the Load

Our team was given a task to create a system that could be used for inspecting manufacturing objects quickly to find out whether they were good for the market or defective, whilst providing proof for that decision.

The team was given three deliverables

- A single function that combines AI classification for PASS/FAIL decisions on non-defective and defective images, respectively, with a score for each decision, with an image processing system that overlays evidence masks and outputs metrics for FAIL-type images. This serves as the inspection system
- A testing script that can divide a dataset into training, validation, and testing sets to be used for evaluating the inspection system and providing results of evaluation through confusion matrices, and yield and defect rates of the system
- A separate test that functions the same as the testing script, but is based on robustness where the system is tested on distorted images from the test set to determine how the system performs, whilst providing extra metrics of accuracy vs false accept and reject rates.

The team began to develop a hierarchical structure of this system:

The system would classify images based on three things:

- Part
- Condition
- Defect Type

Thus, we settled on three classifiers for the job, where they would exist in a group, but be separate from one another. This means we would have to organize the dataset of images in a hierarchical structure, going by part type, condition (pass/fail), and then defect type based on fail images.

The team decided that since these classifiers will output labels that are specific to how the dataset is set up, they can be used for running the algorithms developed by the image processing team for evidence overlay and metric display. The project description gave deliverables for each section of this system, such as having a function that can classify an image and give it a score, as well as a master function that has to do the same thing, but also output evidence overlay for defective images and some chosen metrics associated with them. Because of these deliverables, we were able to develop the following strategy:

- Develop 3 AI classifiers, one for part type, pass/fail (condition), and defect type (if applicable)
- Have the image processing section follow based on the output labels of each classifier. These labels determine which algorithm to run for overlaying and displaying metrics
- Develop the master function that takes an image, classifies it with labels based on the dataset setup (whilst also giving it a score), using those labels to then overlay evidence masks and produce metrics if it is defective

Next, we were assigned to then test the system we developed, in which we determined to use the master function to our advantage

- Create a separate testing script that can be run to evaluate the entire system, and provide metrics for the classifiers based on confusion, yield and defect rates, and false accepts and rejects. A montage of images that we deem to be failure cases of the system will follow.
- Loop the master function inside the testing script to classify the test set
- Do a robustness test using a different, but similar testing script, where the only difference is applying simulated variances such as noise, blur, and contrast changes on all images of the test, and also giving a report on accuracy

The workload between tasks in this live script was split between different members. They are responsible for the implementation, functionality, and presentation of their respective role/task assigned within the team. Each member will demonstrate the points above for their task, while also providing insightful commentary and other information they may deem important. It will be noted in each task which members contributed and what they contributed.

Task 1: Explore and Organize Image Dataset

This task was accomplished by team member: Marc Angarola

Here, the dataset that was chosen and organized by the team is partitioned into a table for later separation for classifier training. We went through a few different datasets, each with their pros (and also many cons). It was fairly troublesome to find a dataset the team could agree to use. There were many object types and many related defects we determined would be too difficult to mask evidence overlays on and also for obtaining a robust network. Some objects had defects that all of us couldn't even notice, and made us question if even a pretrained network would be able to see at all, let alone being able to overlay masks as well. Eventually, we settled on our set. Initially, it contained the following objects:

- Screw
- Grid
- Cable
- Tile
- Toothbrush
- Wood
- Zipper

Both the image processing team and classification team struggled with half these objects, losing valuable time for testing code and obtaining decent training results. Both kept hitting brick walls trying to find out the best formulas for masking and options for training. The dataset constantly shrank and changed, across five versions at that. We kept removing defect types to make the coding easier and training less time-consuming. We eventually settled on the following set

- Grid
- Bottle
- Wood
- Hazelnut

Though our dataset is small, it still has a decent amount of images. There are around 1000 in the entire set, with about 200 -280 passing images and 40 - 70 failing images per object. This is what our team found best to work with across the board.

Below is the code for partitioning the dataset and preparing it for the split to be used for training and testing. It is split based upon the depth and pathing of the set itself. A table is created to hold the Filename (which is the full path directory), the part type, its condition (pass/fail), and the defect label. If the image is considered a PASS, then it receives a label of NONE. FAIL images receive their corresponding labels based on their location in the dataset. This setup makes it easy for the classifier to split the set into many others for training later on.

This was overall fairly simple to implement, with the only difficulty being that certain code that was familiar was nonapplicable here, such as splitLabels, since the table the dataset was incompatible with the syntax. So, "cvpartition" was used, which accomplished the goal. There are issues that could arise with this strategy of separating the dataset, which is that it relies on the pathing of the files, meaning any folders or files added must have the same setup as the others; otherwise, index searching (which is what this section utilizes) will fail.

```
%The following creates a table, where the code can be reused to separate
% labels in other data sets that follow the same structure (part type
% pass/fail, along with the defect types in the fail folder)

%THIS TASK MUST BE RUN IN ORDER TO GET THE REST OF THE SYSTEM TO RUN
% Otherwise, no other task will run

%Insert your directory here (the place in your computer where the dataset
%i s stored) and then click "Run Section" in the toolbar under "SECTION" above to
run Task 1

rawData = "D:\EPP 2026 Dataset\mvtec_anomaly_detection_USED_v5";

imds = imageDatastore(rawData, IncludeSubfolders=true);

%Store text of file names in array
files = string(imds.Files);
elem = numel(files);

parts = strings(elem, 1);
condition = strings(elem, 1);
defectType = strings(elem, 1);

for k = 1:elem
    %Goes through the directory each time and splits objects based on
    %filename
    folderNames = split(files(k), filesep);

    idx = length(folderNames);

    %Assigns label to each image based on folder type
    if folderNames(idx-1) == "PASS"
        parts(k) = folderNames(idx-2);
        condition(k) = "PASS";
        defectType(k) = "NONE";
    else
        parts(k) = folderNames(idx-3);
        condition(k) = "FAIL";
        defectType(k) = folderNames(idx-1);
    end
end
```

```

disp("The table of the entire dataset separated by part, pass/fail, and defect
type")
dataSetTable = table(files, categorical(parts), categorical(condition),...
    categorical(defectType), 'VariableNames', {'Filename', 'Part', 'Part Condition',
'Defect Type'});

disp(dataSetTable)

```

Task 2: Build a Single-Image Inspection Function

Overview

The purpose of the single-image inspection functions is to identify and highlight areas in each image that may contain defects. Different image-processing methods and interpretable metrics were evaluated to determine the best approach for separating defects from the normal features of each part. Since each part and its defects look different, a separate inspection function was created for each part and saved as its own .m file in the `download_matlab_files` folder. Each function produces a binary evidence mask, uses it to create a red evidence overlay, and calculates a few simple metrics describing the suspicious regions found in the image.

Molly

inspectHazelnut.m Function

The `inspectHazelnut` function standardizes each image to 224 x 224 pixels, converts it to grayscale, and applies mild Gaussian denoising. A fixed region of interest is used to focus on the hazelnut because it originally occupied only part of the full image. Cropping will reduce background interference and allow the segmentation to focus more closely on the hazelnut. Because the defects have different appearances, separate segmentation branches are used: `crack` and `hole_hazel` are treated as dark defects, while `cut` and `print` are treated as bright defects. Adaptive thresholding and morphological operations are used to reduce normal texture, exclude the outer shell boundary, and connect disconnected defect regions. The final evidence mask is displayed as a red overlay and used to calculate the number of suspicious regions, largest region area, and area ratio.

The hazelnut inspection function identified suspicious regions for all four defect types. Depending on the image, a crack could be detected as one continuous region, or as several separated regions. The hole and cut evidence also produced multiple highlighted areas, some of which may be false detections caused by shell texture or the hazelnut border. Overall, the segmentation program successfully highlighted the main defect regions, although some false detections remained.

One challenge was selecting morphological operations that worked well across multiple dark hazelnut defect types. I tested `imfill`, but decided not to include it in the dark-defect segmentation branch because it tended to

overflow the detected regions and overflow the hole defect. This caused the evidence mask to become too large and less representative of the actual defect.

Another challenge was developing the segmentation branch for the cut and print defects. The initial thresholding method repeatedly highlighted the outer border of the hazelnut instead of only the defect. To reduce this false detection, a rough mask of the hazelnut was created and eroded to define a smaller inner region. This excluded most of the outer shell boundary and allowed the final evidence mask to focus more closely on the actual defect.

Two different segmentation branches were necessary because one thresholding method did not reliably detect both dark and bright defects. The crop dimensions, threshold sensitivities, and morphological settings were adjusted using sample images from each hazelnut defect category to improve defect detection while reducing false detections from the shell texture and the object outline.

inspectBottle.m Function

The inspectBottle function standardizes each image to 224 x 224 pixels, converts it to grayscale, and applies mild Gaussian denoising. Because the bottle is centered against a plain background, the full image is used as the region of interest. A circular ring mask is then created to limit the analysis to the bottle-rim area, where the broken defects are located. Local standard deviation is used to highlight areas with greater intensity variation, since damaged regions were expected to appear more irregular than smoother, undamaged areas. Adaptive thresholding and morphological operations are then used to create the evidence mask and connect nearby suspicious regions.

The bottle inspection function detected suspicious regions in both the broken-large and broken-small images. However, a lot of the inner bottle rim was also highlighted because its strong edges, reflections, and texture produced intensity variation similar to the actual broken areas. As a result, the size and number of highlighted regions varied between images. The evidence metrics were useful for describing the generated mask, but they did not consistently represent the true defect size. Overall, the bottle was the most difficult part to segment because the damaged areas had texture and edge characteristics similar to the normal rim.

The challenge for the bottle images was separating the broken defect from the normal inner rim. The `stdfilt` method successfully detected irregular texture, but it also responded strongly to the rim's natural edges and reflections. A fixed ring mask reduced detections outside the expected defect area, but it could not fully distinguish the broken region from the normal rim. This demonstrates a limitation of using texture-based thresholding alone for the bottle images.

inspectWood.m Function

The inspectWood function standardizes each image to 224 × 224 pixels. Because the wood surface fills the image, the full image is used as the region of interest. The RGB image is converted to Lab color space, which separates lightness from color information. The lightness channel contains one brightness value per pixel, so

it can be displayed and processed as a grayscale intensity image. This channel is used instead of `rgb2gray` because it isolates brightness while reducing the influence of the wood's natural brown color variation. The lightness values are then scaled from 0 to 1 and mildly smoothed to reduce small wood-grain fluctuations and noise. Focusing on lightness allows MATLAB to search for areas that appear unusually dark or bright compared with the surrounding wood surface.

Separate segmentation branches are used because the wood defects have different appearances. The `hole_wood` and liquid defects are treated as dark regions, while scratches are treated as bright regions. Adaptive thresholding finds areas that are darker or brighter than the wood around them. Morphological opening is then used to reduce long wood-grain responses, area filtering removes small isolated regions, and morphological closing connects nearby fragments that may belong to the same defect.

The wood inspection function detected the main defect regions in the hole, liquid, and scratch examples. The hole images produced the best overall results because the evidence masks focused mainly on the actual defect regions. Morphological opening was especially helpful for the hole defects because it removed most of the long vertical wood-grain responses that were falsely highlighted. A few false detections were still present, but they were less noticeable than those in the liquid and scratch images. As a result, the hole images generally produced fewer suspicious regions. The liquid and scratch images were more difficult because many natural wood-grain patterns were also highlighted, increasing both the number and size of the detected regions.

One challenge was selecting threshold values that could detect both thin and thick scratches. Threshold settings that worked well for thicker scratches sometimes failed to detect thinner scratches. Therefore, a middle-ground value was selected so the function could identify most scratch defects, although some thin scratches were still missed.

Another challenge was separating actual defects from the natural grain pattern. The grain contains long bright and dark lines that can resemble liquid streaks or scratches. Using separate dark- and bright-defect branches improved the results, but the scratch images still produced many false detections from normal grain lines. This shows that intensity-based thresholding alone cannot always distinguish defects from normal wood texture. Overall, the function usually located the main suspicious areas, but the liquid and scratch masks still contained noticeable false detections, and some parts of the actual defects were not always highlighted.

Basil

inspectGrid.m function

The `inspectGrid` function uses the area, median, and std deviation of the images in order to pin-point where the defect is. For a grid, the holes are the main indicator on how healthy the image is, when a hole seems smaller or larger than usual, chances are it's a defect.

The basic thinking process when deciding on how I should find where the defect was went something like this:

- When I look at the binarized image next to the raw image, what differences do I notice?
- Can I utilize these differences and isolate them mathematically? Are they less circular? Are the defects smaller in size? Maybe bigger in size?

For metal contamination (Where there's a piece of metal on the grid, covering some of the holes) and the glue defect, a straight forward approach was to see which holes are relatively smaller, and mask those as defects. Relatively here is an important word, because in the beginning, I used hard coded numbers by checking the matrices. One of the ones I often used was `stats.Area`, I would check the `max(stats.Area)`, `min`, `median`, `mean`, and I would also simply scroll through it and see what numbers seem odd. This really helped in understanding what's wrong with the defect, but when I used these numbers as thresholds (for example, remove any blob > 4500) they didn't work well across all images, this is because of different orientations, some of the photo being cut out, etc..

My next step was to use the average(mean) of the areas of these images,.. this also proved to be faulty and didn't work as well as I wished it would. The reason why I believe this did not work, is because the mean of a photo can be heavily skewed by multiple outliers. To explain simply, if we had an image of 5 same size holes, but then 3 small size holes, the 3 small size holes will have a strong weight on the average.

I then discovered that the median is MUCH more helpful and safe. The median area (in this case, the median hole) will almost always be the average sized hole. We can then compare everything else to this median hole!

Then, even better, I found the `mad()` function(Median absolute deviation) which works even BETTER! This will use the median that I found, let's say 80, and subtracts every area from that number to find the deviation, and then outputs the median of the deviated numbers! This makes it very robust.

Quick ex: [50 50 50 50 10 10 10]

Median = 50

Now we find the deviation from each area.

[0 0 0 0 40 40 40] (This is always sorted)

now the mad is = 0.

Now I can use the mad as a threshold for all of my images.

Note: Also a big issue was the edges of the image, some of the holes at the very edges were cut off, making them much smaller. To combat this, I used `imclearborder()` to get rid of the border holes.

Issues and challenges:

I struggled with the screw for a long time. The threading was a big issue and I could not separate the threading mask from the scratches. I spent lots of time on this and unfortunately I was not proud of the results and decided to not include the function. Although the function was clearly showing the defect, it also mistakenly identified the threads as a defect consistently, masking every passing image's screw threading as a defect.

Another one I had issues with was the thread defect from the grid data set. I did not use specific metrics to isolate the thread, but I instead simply used morphological operations such as closing and opening to try and isolate most of the thread. This worked on almost all of the images, but on one or two it did not work as well as I wanted it to.

Results and a brief reflection:

I was very proud of the results, and a lot of this honestly taught me a lot. I would spend hours on trying to create a working segmentation code for a specific defect, then out of no where, I mistakenly used another defect's segmentation code for it.. and it actually does a good job! Of course I would have to tweak a couple of numbers here and there, but these "happy accidents" really pointed me in the right direction.

Some of the segmentation was also really simpler than it looks. Most of them being able to be done with only a couple of lines of code. (Although I did find it helpful to use different preprocessing on some of the defects)

Whenever I look at any object now, I ask myself why I know that I'm looking at a cup or a TV. It's because of its features! I know it's a TV because it has small legs and a large rectangular shape, and I know that another object is a cup because of its handle and the circular opening at the top. Although we use color to help us distinguish objects sometimes, most things (if not all) can be distinguished with just binary features, and this project really opened my eyes to that.

```
clear;
clc;
close all;

% ----- HAZELNUT
% -----
% NOTE: hz = hazelnut

% Use the Import Data Live Tasks below to select one image from each hazelnut
% defect category.
% The imported variables should be named: hz_crackImg, hz_holeImg, hz_cutImg, and
% hz_printImg.

% If any required image or variable is missing, the section will not run properly.
% Import image
```

```

hz_crackImg =
imread(".\mvtec_anomaly_detection_USED_v4\hazelnut\FAIL\crack\001.png");

% Display results
imshow(hz_crackImg);
title("Image Data from 001.png");
% Import image
hz_cutImg = imread(".\mvtec_anomaly_detection_USED_v4\hazelnut\FAIL\cut\000.png");

% Display results
imshow(hz_cutImg);
title("Image Data from 000.png");
% Import image
hz_holeImg =
imread(".\mvtec_anomaly_detection_USED_v4\hazelnut\FAIL\hole_hazel\000.png");

% Display results
imshow(hz_holeImg);
title("Image Data from 000.png");
% Import image
hz_printImg =
imread(".\mvtec_anomaly_detection_USED_v4\hazelnut\FAIL\print\000.png");

% Display results
imshow(hz_printImg);
title("Image Data from 000.png");

% Pass each defect label so the function selects the appropriate dark- or bright-
defect segmentation branch.
[hz_crackMask, hz_crackEvidence, hz_crackROI, hz_crackROI_RGB, hz_crackOverlay] =
inspectHazelnut(hz_crackImg, "crack");
[hz_holeMask, hz_holeEvidence, hz_holeROI, hz_holeROI_RGB, hz_holeOverlay] =
inspectHazelnut(hz_holeImg, "hole_hazel");

[hz_cutMask, hz_cutEvidence, hz_cutROI, hz_cutROI_RGB, hz_cutOverlay] =
inspectHazelnut(hz_cutImg, "cut");
[hz_printMask, hz_printEvidence, hz_printROI, hz_printROI_RGB, hz_printOverlay] =
inspectHazelnut(hz_printImg, "print");

% Crack overlay.
figure;
imshow(hz_crackOverlay);
title("Hazelnut Crack: Red Evidence Overlay");

% Hole overlay.
figure;
imshow(hz_holeOverlay);
title("Hazelnut Hole: Red Evidence Overlay");

% Cut overlay. Hazelnut crack metrics:

```

```

figure;
imshow(hz_cutOverlay);
title("Hazelnut Cut: Red Evidence Overlay");

% Print overlay.
figure;
imshow(hz_printOverlay);
title("Hazelnut Print: Red Evidence Overlay");

% Interpretable metrics for crack, hole, cut, & print:

fprintf("\nHazelnut crack metrics:\n");
fprintf("Number of suspicious regions: %d\n", hz_crackEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n", hz_crackEvidence.maxArea);
fprintf("Area ratio: %.4f\n", hz_crackEvidence.areaRatio);

fprintf("\nHazelnut hole metrics:\n");
fprintf("Number of suspicious regions: %d\n", hz_holeEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n", hz_holeEvidence.maxArea);
fprintf("Area ratio: %.4f\n", hz_holeEvidence.areaRatio);

fprintf("\nHazelnut cut metrics:\n");
fprintf("Number of suspicious regions: %d\n", hz_cutEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n", hz_cutEvidence.maxArea);
fprintf("Area ratio: %.4f\n", hz_cutEvidence.areaRatio);

fprintf("\nHazelnut print metrics:\n");
fprintf("Number of suspicious regions: %d\n", hz_printEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n", hz_printEvidence.maxArea);
fprintf("Area ratio: %.4f\n", hz_printEvidence.areaRatio);

```

```

clear;
clc;
close all;

% ----- BOTTLE
% -----
% --

% NOTE: bo = bottle

% Use the Import Data Live Tasks below to select one image from each bottle defect
category.
% The imported variables should be named: bo_largeImg and bo_smallImg.

% If any required image or variable is missing, the section will not run properly.
% Import image
bo_largeImg =
imread(".\mvtec_anomaly_detection_USED_v4\bottle\FAIL\broken_large\002.png");

```

```

% Display results
imshow(bo_largeImg);
title("Image Data from 002.png");
% Import image
bo_smallImg =
imread("./mvtec_anomaly_detection_USED_v4\bottle\FAIL\broken_small\000.png");

% Display results
imshow(bo_smallImg);
title("Image Data from 000.png");

% Apply the same bottle inspection function to both bottle defect types.
[bo_largeMask, bo_largeEvidence, bo_largeROI, bo_largeROI_RGB, bo_largeOverlay] =
inspectBottle(bo_largeImg);
[bo_smallMask, bo_smallEvidence, bo_smallROI, bo_smallROI_RGB, bo_smallOverlay] =
inspectBottle(bo_smallImg);

% Broken_large overlay.
figure;
imshow(bo_largeOverlay);
title("Bottle Broken Large: Red Evidence Overlay");

% Broken_small overlay.
figure;
imshow(bo_smallOverlay);
title("Bottle Broken Small: Red Evidence Overlay");

fprintf("\nBottle broken_large metrics:\n");
fprintf("Number of suspicious regions: %d\n", bo_largeEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n", bo_largeEvidence.maxArea);
fprintf("Area ratio: %.4f\n", bo_largeEvidence.areaRatio);

fprintf("\nBottle broken_small metrics:\n");
fprintf("Number of suspicious regions: %d\n", bo_smallEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n", bo_smallEvidence.maxArea);
fprintf("Area ratio: %.4f\n", bo_smallEvidence.areaRatio);

```

```

clear;
clc;
close all;

```

```

% ----- WOOD
% -----
%
% NOTE: wo = wood

```

```

% Use the Import Data Live Tasks below to select one image from each wood defect
category.
% The imported variables should be named: wo_holeImg, wo_liquidImg, and
wo_scratchImg.

% If any required image or variable is missing, the section will not run properly.
% Import image
wo_holeImg =
imread(".\mvtec_anomaly_detection_USED_v4\wood\FAIL\hole_wood\003.png");

% Display results
imshow(wo_holeImg);
title("Image Data from 003.png");
% Import image
wo_liquidImg = imread(".\mvtec_anomaly_detection_USED_v4\wood\FAIL\liquid\000.png");

% Display results
imshow(wo_liquidImg);
title("Image Data from 000.png");
% Import image
wo_scratchImg =
imread(".\mvtec_anomaly_detection_USED_v4\wood\FAIL\scratch\000.png");

% Display results
imshow(wo_scratchImg);
title("Image Data from 000.png");

% Pass each defect label so the function selects the appropriate dark- or bright-
defect segmentation branch.
[wo_holeMask, wo_holeEvidence, wo_holeROI, wo_holeROI_RGB, wo_holeOverlay] =
inspectWood(wo_holeImg, "hole_wood");
[wo_liquidMask, wo_liquidEvidence, wo_liquidROI, wo_liquidROI_RGB, wo_liquidOverlay]
= inspectWood(wo_liquidImg, "liquid");

[wo_scratchMask, wo_scratchEvidence, wo_scratchROI, wo_scratchROI_RGB,
wo_scratchOverlay] = inspectWood(wo_scratchImg, "scratch");

% Hole overlay.
figure;
imshow(wo_holeOverlay);
title("Wood Hole: Red Evidence Overlay");

% Liquid overlay.
figure;
imshow(wo_liquidOverlay);
title("Wood Liquid: Red Evidence Overlay");

% Scratch overlay.
figure;
imshow(wo_scratchOverlay);

```

```

title("Wood Scratch: Red Evidence Overlay");

fprintf("\nWood hole metrics:\n");
fprintf("Number of suspicious regions: %d\n", wo_holeEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n", wo_holeEvidence.maxArea);
fprintf("Area ratio: %.4f\n", wo_holeEvidence.areaRatio);

fprintf("\nWood liquid metrics:\n");
fprintf("Number of suspicious regions: %d\n", wo_liquidEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n",
wo_liquidEvidence.maxArea);
fprintf("Area ratio: %.4f\n", wo_liquidEvidence.areaRatio);

fprintf("\nWood scratch metrics:\n");
fprintf("Number of suspicious regions: %d\n", wo_scratchEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n",
wo_scratchEvidence.maxArea);
fprintf("Area ratio: %.4f\n", wo_scratchEvidence.areaRatio);

```

```

clear;
clc;
close all;

% ----- Grid
% -----

% Use the Import Data Live Tasks below to select one image from each grid defect
category.
% The imported variables should be named: gr_bentImg, gr_brokenImg, gr_glueImg,
gr_metalContImg, gr_threadImg.

% If any required image or variable is missing, the section will not run properly.
% Import image
gr_bentImg = imread(".\mvtec_anomaly_detection_USED_v4\grid\FAIL\bent\001.png");

% Display results
imshow(gr_bentImg);
title("Image Data from 001.png");
% Import image
gr_brokenImg = imread(".\mvtec_anomaly_detection_USED_v4\grid\FAIL\broken\000.png");

% Display results
imshow(gr_brokenImg);
title("Image Data from 000.png");
% Import image
gr_glueImg = imread(".\mvtec_anomaly_detection_USED_v4\grid\FAIL\glue\000.png");

```

```

% Display results
imshow(gr_glueImg);
title("Image Data from 000.png");
% Import image
gr_metalContImg =
imread("..\mvtec_anomaly_detection_USED_v4\grid\FAIL\metal_contamination\000.png");

% Display results
imshow(gr_metalContImg);
title("Image Data from 000.png");
% Import image
gr_threadImg = imread("..\mvtec_anomaly_detection_USED_v4\grid\FAIL\thread\002.png");

% Display results
imshow(gr_threadImg);
title("Image Data from 002.png");

% Pass each defect label so the function selects the appropriate defect branch.
[gr_bentEvidence, gr_bentOverlay] = inspectGrid(gr_bentImg, "bent");
[gr_brokenEvidence, gr_brokenOverlay] = inspectGrid(gr_brokenImg, "broken");
[gr_glueEvidence, gr_glueOverlay] = inspectGrid(gr_glueImg, "glue");
[gr_metalContEvidence, gr_metalContOverlay] = inspectGrid(gr_metalContImg,
"metal_contamination");
[gr_threadEvidence, gr_threadOverlay] = inspectGrid(gr_threadImg, "thread");

% Bent overlay.
figure;
imshow(gr_bentOverlay);
title("Grid Bent: Red Evidence Overlay");

% Broken overlay.
figure;
imshow(gr_brokenOverlay);
title("Grid Broken: Red Evidence Overlay");

% Glue overlay.
figure;
imshow(gr_glueOverlay);
title("Grid Glue: Red Evidence Overlay");

% Metal_contamination overlay.
figure;
imshow(gr_metalContOverlay);
title("Grid Metal Contamination: Red Evidence Overlay");

% Thread overlay.
figure;
imshow(gr_threadOverlay);
title("Grid Thread: Red Evidence Overlay");

```



```

fprintf("\nGrid bent metrics:\n");
fprintf("Number of suspicious regions: %d\n", gr_bentEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n", gr_bentEvidence.maxArea);
fprintf("Area ratio: %.4f\n", gr_bentEvidence.areaRatio);

fprintf("\nGrid broken metrics:\n");
fprintf("Number of suspicious regions: %d\n", gr_brokenEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n",
gr_brokenEvidence.maxArea);
fprintf("Area ratio: %.4f\n", gr_brokenEvidence.areaRatio);

fprintf("\nGrid glue metrics:\n");
fprintf("Number of suspicious regions: %d\n", gr_glueEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n", gr_glueEvidence.maxArea);
fprintf("Area ratio: %.4f\n", gr_glueEvidence.areaRatio);

fprintf("\nGrid metal contamination metrics:\n");
fprintf("Number of suspicious regions: %d\n", gr_metalContEvidence.numComponents);
fprintf("Size of largest suspicious region: %d pixels\n",
gr_metalContEvidence.maxArea);
fprintf("Area ratio: %.4f\n", gr_metalContEvidence.areaRatio);

```

Task 3: Train and Integrate an AI Classifier into your Single-Image Inspection Function

Contributing Members: Marc Angarola

Background

Classification is a process where things are separated based on a label. To be able to know how a label fits something, one must understand why that thing has that label. The why is called a feature, which gives distinction to the label. Artificial Intelligence can be used to extract features from something and learn to classify things based on those features. There is a deeper analysis as to why and how these models learn features and are able to associate labels to them, but based on the scale of this project and to keep explanations to the point, those details can be omitted.

Within MATLAB, there are resources and tools that give users access to pre-constructed AI classification models, as well as tools for building a classifier from the ground up. For the purpose of this project, pre-constructed classification models are used. They have been trained (they have learned) to classify (assign labels) objects on a multitude of different images, regardless of background, noise, filters, etc. They extract those key features that are associated with a label. These models consist of layers, which are fine-tuned to adjust what features are key and which are not based on their position. This makeup of layers establishes a network, which is a system consisting of many different parts that work together to accomplish a task. In this case, finding out what makes something, something. Different networks are trained based on these layers and what they are tuned

to extract from data. This data can be images, which is what the purpose of this section is: extract useful features from images needed to classify images based on labels assigned to them.

Though this task is after the image processing section, it actually comes first. This is because the evidence overlaying algorithm based on the teams setup needs the labels from the part and defect type classifier to run their correct masking procedure on an image

The AI Classifier

The code below begins with storing the table from Task 1 into a datastore, which is a function in MATLAB that stores all the key information about a file folder. This datastore is split using MATLAB functions to establish training, validation, and testing sets. The network should see most of the images; that way it has the most data to extract the needed information for associating a label. Validation is used to see if the network is truly learning, since it shows it images it has never seen before, proving whether or not it's finding features that are similar to other images of the same type.

A pretrained network is called from the Deep Learning Toolbox in MATLAB. Based on the images in the dataset (and recommendations from the project description), resnet18 was chosen. Resnet18 has seen many images and has 18 fine-tuned layers for recognizing many objects. Thus, it can train quickly and maintain efficiency based on the chosen dataset the team has declared to use.

Every classifier has four networks, which are thanks to the object types in the dataset. There is a network for each object type regarding pass/fail classification and defect type. The part classifier is a single network since it has access to the whole dataset for the split needed for training and validation and only classifies on object type, which are the main subfolders of the dataset.

The structure system allows the key characteristics of each classifier to be stored close to one another for easier access in the final master function, which will combine the single image processing functions, and the deliverable for this section, which is a function that takes the network structure as input and a single image.

The structure of networks also plays a role for other functions that are utilized, such as one in the testing script to manually train one's own set of networks for classification and then have it evaluated based on performance and robustness. This would be "classifyTestSuite," which is similar to the code below, but different.

Struggles

A lot of struggles came from that one (classifyTestSuite), particularly with partitioning the data, as the code below is drastically different than in "classifyTestSuite" for it. The data splits kept missing defect types when they were split, causing errors to occur during training since the number of labels in a training set and validation set wouldn't match since it did not exist in one set. That took a while to figure out, and valuable time was lost trying to make it work.

The biggest struggle for the AI classifiers turned out to be not writing, but training. The part classifier always performed well, as it had many images to train and the pretrained weights it had worked perfectly with the objects we chose. Condition classification was worse than the part classifier, but still reasonable, as the available images were still at a decent amount and had a good split between training and validation. However, the defect classifier was and is still the worst. The reason for it is really just the amount of images it has to work with.

Only around maybe 40 - 80 images were available per part, as this classifier only looks at the defect types and not pass images. This is something the weights of ResNet18 were not used to, and it is not good practice to train on a small dataset of images for any network. However, our methodology was already planned out, and so I had spent a lot of time adjusting training options for each. I used all the resources I could, including LLM Artificial Intelligence. Eventually, options that proved sufficient were discovered for each network based on the dataset sizes.

It is something to note that since the splits are random every time, a lot of time was spent retraining for the condition and defect type classifier. Some objects performed better than others; most notably, the worst of them all was the grid object. Rarely did a training session bring back decent metrics after multiple attempts. This includes not just the defect type label but also the condition classifier too. There must be something about the structure of the image that causes ResNet18 to not pull any key features.

The section below can be run to see how a network trains based on the notes within the coded section

(Deliverable at the bottom of section): This classifies a single image

Deliverable: [aiLabel, aiScore] = classify(net, roiForNet)

```
%Make sure to run task 1 before running this section

%The following is a setup of the AI classifier. Running the section below
% will train each network based on the latest version of the
% dataset from Task 1 (no testing split, only training and validation).

% ONLY RUN IF YOU WISH TO TRAIN THE NETWORK. If not, there is a variable
% called "testingScriptNetworksAndClasses.mat", which is a structure of already
% trained
% networks that are ready for use. The loading of the variable will be
% included at separate sections for you to continue moving through the
% live script, so there is no need for you to do it manually (unless you
% want to)

%This section is meant to showcase that the code works for training and
% validation based on the setup of the dataset that was partitioned in Task 1.
%A separate function will be established that trains each network based on
```

```

% a initial separation of the dataset into a train, validation, and test
% set (this is classifyTestSuite.m). It is similar to the code below, but
% different in some aspects, since it is a function rather than just a
% script.

%Note: The networks used for classifying part type and defect type are
% used for determining which algorithms to run within single image
% function. They play no role for what the output of the system classifier is, but
do
% play a role in outputting the correct evidence mask

%This section also displays sanity checks for the training of each network
% for proof that each network is being trained and validated on the correct images
% for each network (DISPLAY INFO marks these sections)

%-----Network 1: Part Classifier Network

%Create structure for classifiers (will store networks and classes labels
% and the name of each classifier

InspectionSystem = struct;

%Create separate datastore to be used for network 1
imdsClassify = imageDatastore(dataSetTable.Filename);
imdsClassify.Labels = dataSetTable.Part;
partLabels = imdsClassify.Labels

%Split data from table into validation and training and shuffle seeding for
% for images to be picked for splitting
rng("shuffle")
labelSplit = cvpartition(partLabels, "Holdout", 0.20, 'Stratify',true);

%gives logical indices where training and validation elements are
trainIdx = training(labelSplit);
valIdx = test(labelSplit);

%Place data into a subset datastore for training and validation
imdsTrain = subset(imdsClassify, trainIdx);
imdsVal = subset(imdsClassify, valIdx);

%Load pretrained network and create and modify final layer for multiple
%outputs
classes = numel(categories(partLabels));
netPart = imagePretrainedNetwork("resnet18", NumClasses=classes);

%Resize images to match the network and randomly modify images to avoid
%overfitting during learning
inputSize = netPart.Layers(1).InputSize(1:2);

```

```

partAugmenter = imageDataAugmenter(RandRotation=[-180 180]);

partAugmentTrain = augmentedImageDatastore(inputSize, imdsTrain,
DataAugmentation=partAugmenter, ColorPreprocessing="gray2rgb");
partAugmentVal = augmentedImageDatastore(inputSize, imdsVal,
ColorPreprocessing="gray2rgb");

%Modify training options for Part network
partOptions = trainingOptions("adam", InitialLearnRate=3e-4,...
    MaxEpochs=20, MiniBatchSize=16, Shuffle="every-epoch",...
    ValidationData=partAugmentVal, ValidationPatience=8,...
    ValidationFrequency=10, Plots="training-progress",...
    L2Regularization=1e-4, OutputNetwork="best-validation",...
    Metrics="accuracy", Verbose=false);

%-----DISPLAY INFO-----

%Check labels and number of files to be used during training(ensure they match
dataset setup)
disp(categories(imdsTrain.Labels))
countEachLabel(imdsTrain)
numel(imdsTrain.Files)

disp(categories(imdsVal.Labels))
countEachLabel(imdsVal)
numel(imdsVal.Files)

%Compare above lines of code display to ensure output matches what is in
% original table (and the math makes sense based on the split)
numel(imdsClassify.Files)
height(dataSetTable)

%-----END DISPLAY INFO-----

%Train network
fprintf("Training Parts Classifier...")
trainedPartClassifier = trainnet(partAugmentTrain, netPart, "crossentropy",
partOptions);

%Add to structure of networks (InspectionSystem)
InspectionSystem.PartClassifier = struct;
InspectionSystem.PartClassifier.Network = trainedPartClassifier;
InspectionSystem.PartClassifier.Classes = categories(partLabels);
InspectionSystem.PartClassifier.Name = "Part Classifier";

%----Network 2: Condition Classifier

```

```

%Separate part types
partsType = categories(dataSetTable.Part);

%Create structure for condition classifiers
InspectionSystem.ConditionClassifiers = struct;

%Loop to train each network based on part type
for k = 1:numel(partsType)

    %Converts labels to usable strings for classifier
    % structure access

    partName = matlab.lang.makeValidName(partsType{k});

    fprintf("Training %s...\n", partsType{k})
    InspectionSystem.ConditionClassifiers.(partName) =...
        trainConditionClassifier(dataSetTable, partsType{k});
end

%Function that will train pass/fail networks for each part in dataset
function conditionClassifier = trainConditionClassifier(dataSetTable, partName)

%The following will be used to establish the train and validation sets (use
% outside in testing script, but make sure to keep the code for: KEEP)
    %Determine part to be trained
    idx = dataSetTable.Part == partName;

    %Establish table for part to be trained
    partTable = dataSetTable(idx,:);

%-----DISPLAY INFO-----

    %Display total fail and pass images for each part before training (ensure it
    matches
    % amount in dataset)
    fprintf("\n%s\n", partName)
    disp(categories(partTable.Part))
    countEachLabel(imageDatastore(partTable.Filename, Labels=partTable.("Part
    Condition")))

%-----END DISPLAY INFO-----

    %Image datastore for part
    imds = imageDatastore(partTable.Filename);
    imds.Labels = partTable.("Part Condition");

```

```

%Split data for training and validation
dataSplit = cvpartition(imds.Labels, "Holdout", 0.20, 'Stratify',true);
trainImds = subset(imds, training(dataSplit));
valImds = subset(imds, test(dataSplit));

conditionClasses = 2;

%Load pretrained network
netDefect = imagePretrainedNetwork("resnet18", "NumClasses", conditionClasses);

%Manage input size for network
conditionSize = netDefect.Layers(1).InputSize(1:2);

%Resize images and apply random modifications to avoid overfitting
defectAugmenter = imageDataAugmenter(RandRotation=[-180 180]);

conditionAugmentTrain = augmentedImageDatastore(conditionSize, trainImds,
DataAugmentation=defectAugmenter,...
    ColorPreprocessing="gray2rgb");

conditionAugmentVal = augmentedImageDatastore(conditionSize, valImds,
ColorPreprocessing="gray2rgb");

%Define training options
defectOptions = trainingOptions("adam", InitialLearnRate=1e-4,...
    MaxEpochs=25, MiniBatchSize=8,...
    ValidationData=conditionAugmentVal, Shuffle="every-epoch",...
    ValidationFrequency=5, ValidationPatience=15,...
    Plots="training-progress",L2Regularization=1e-3,...
    OutputNetwork="best-validation", Metrics="accuracy",...
    Verbose=false);

%Train network
defectNetTrained = trainnet(conditionAugmentTrain, netDefect, "crossentropy",
defectOptions);

%Store relevant pieces into condition classifier
conditionClassifier.Network = defectNetTrained;
conditionClassifier.Classes = categories(imds.Labels);
conditionClassifier.Name = string(partName) + " Condition Classifier";

end

%}

%----- Network 3: Defect Classification (used by image processing in case
%         of fail)

```

```

% Create a structure for defect type classifiers
InspectionSystem.DefectTypeClassifiers = struct;

% Train every network and store it
for k = 1:numel(partsType)

    %Converts labels to usable strings for classifier
    % structure access

    partName = matlab.lang.makeValidName(partsType{k});

    fprintf("Training %s Defect-Type Network...\n",partsType{k})
    InspectionSystem.DefectTypeClassifiers.(partName) =...
        trainDefectTypeClassifier(dataSetTable,partsType{k});

end

% Function that will train every defect type network (loops from loop above)
function typeClassifier = trainDefectTypeClassifier(dataSetTable,partName)

    %Create table for specified part
    idxPart = dataSetTable.Part == partName;
    partTable = dataSetTable(idxPart,:);

    %Create table for FAIL images of part
    failIdx = partTable("Part Condition") == "FAIL";
    failTable = partTable(failIdx,:);

    %Remove labels that are not found in within an objects fail folder
    failTable("Defect Type") = removecats(failTable("Defect Type"));

%-----DISPLAY INFO-----

    %Display part that is to be trained along with the amount of images in
    % each fail subfolder (defect type images)
    disp(unique(failTable.Part))
    disp(categories(failTable("Defect Type")))
    countEachLabel(imageDatastore(failTable.FileName, Labels=failTable("Defect
Type")))

%-----END DISPLAY INFO-----

    %Create datastore to establish fail labels to define the fails for the part
    imds = imageDatastore(failTable.FileName);
    imds.Labels = failTable("Defect Type");

    %Split data for training and validation
    splitData = cvpartition(imds.Labels, "HoldOut", 0.2, 'Stratify', true);

```



```

trainData = subset(imds, training(splitData));
valData = subset(imds, test(splitData));

%Create classes for defect network (based on number of defect type for
%an object
typeClasses = numel(categories(imds.Labels));
typeNetwork = imagePretrainedNetwork("resnet18", "NumClasses", typeClasses);

%Establish input size for network model
typeSize = typeNetwork.Layers(1).InputSize(1:2);

%Augment Data for training and validation and train network

typeAugmenter = imageDataAugmenter(RandRotation=[-180 180]);
augmentedTrain = augmentedImageDatastore(typeSize, trainData,
>DataAugmentation", typeAugmenter,...
    "ColorPreprocessing", "gray2rgb");
typeAugmentVal = augmentedImageDatastore(typeSize, valData,
"ColorPreprocessing", "gray2rgb");

options = trainingOptions("adam", InitialLearnRate=5e-4,...
    MaxEpochs=120, MiniBatchSize=16, ValidationData=typeAugmentVal,...
    Shuffle="every-epoch", ValidationFrequency=2,...
    Plots="training-progress", L2Regularization=1e-4,...
    OutputNetwork="last-iteration", Metrics="accuracy", Verbose=false);

defectTypeNet = trainnet(augmentedTrain, typeNetwork, "crossentropy", options);

% Add parts of network to structure
typeClassifier.Network = defectTypeNet;
typeClassifier.Classes = categories(imds.Labels);
typeClassifier.Name = string(partName) + " Defect Type Classifier";

end

%----The following is a function used for inputting a single
% image into each network for classification.

%DELIVERABLE
% Function used for classifying a SINGLE image
function [aiLabel, aiConfidence] = classifyImg(img, classifierStruct)
    %Reassign img to I
    I = img;

    %Use code to convert grayscale to RGB if needed
    if size(I,3) == 1
        I = repmat(I,[1 1 3]);
    end
end

```

```

%Determine input size needed
neededSize = classifierStruct.Network.Layers(1).InputSize(1:2);

%Resize Image
I = imresize(I, neededSize);

%Convert image to array format for network
IArray = dlarray(single(I), "SSCB");

%Obtain scores
aiScores = minibatchpredict(classifierStruct.Network, IArray);

%Use if needed to convert to MATLAB array
%{
aiScores = extractdata(aiScores);
%}

%Find highest score for label output
[aiConfidence, idx] = max(aiScores);

%Obtain label
aiLabel = classifierStruct.Classes(idx);
end

```

Task 3 Function: Testing the Deliverable function

Contributing Members: Marc Angarola

Here, the deliverable can be tested based on two different sections. The deliverable is a function that utilizes a trained network structure (Task 3) to classify an image. This function is reusable for all different classifiers. More details are explained in the code below.

```

%THIS SECTION IS TO BE RAN IF THE USER HAS TRAINED THEIR OWN NETWORKS USING
%TASK 3. DO NOT RUN THIS SECTION IF YOU HAVE NOT TRAINED YOUR OWN NETWORKS

%Insert an image of your choice
% (change your file type to "All Files (*.*)" when your window opens (bottom right
corner of it))

I = "D:\EPP 2026 Dataset\mvtec_anomaly_detection_USED_v5\wood\Fail\liquid\009.png";
imageChoice = imread(I);

%Run image classification for part type (needed to access pass/fail
% networks)
[partLabel, partConfidence] = classifyImg(imageChoice,
InspectionSystem.PartClassifier);

```

```

%Run image classification function to return label and confidence score for
% PASS/FAIL
[imgLabel, imgConfidence] = classifyImg(imageChoice,...
    InspectionSystem.ConditionClassifiers.(string(partLabel)));

%Show image along with AI classification results
figure

imshow(imageChoice)
title("Your chosen image")
fprintf("Final Condition Classifier label: %s\n", string(imgLabel));
fprintf("Condition Classifier Confidence score: %.2f%%\n", imgConfidence * 100);

fprintf("Final Part Classifier label: %s\n", string(partLabel));
fprintf("Part Classifier Confidence score: %.2f%%\n", partConfidence * 100);

if lower(string(imgLabel)) == "fail"
    %Run image classification function to return label and confidence score for
    % defect type
    [defectLabel, defectConfidence] = classifyImg(imageChoice,
InspectionSystem.DefectTypeClassifiers.(string(partLabel)));
    fprintf("Final Defect Classifier label: %s\n", string(defectLabel));
    fprintf("Defect Classifier Confidence score: %.2f%%\n", defectConfidence * 100);

else
    return
end

```

```

%THIS SECTION IS TO BE RAN IF THE USER HAS NOT TRAINED THEIR OWN NETWORKS
USING
%TASK 3. DO NOT RUN THIS SECTION IF YOU HAVE TRAINED YOUR OWN NETWORKS

%Load in ready for classification networks from variable
% (an error may occur since the section will run in its entirety
% Just select the correct image directory at "Click here to insert image"
% and the section should run fine)
load testingScriptNetworksAndClasses.mat

%Insert an image of your choice
% (change your file type to "All Files (*.*)" when your window opens (bottom right
corner of it))

I = "D:\EPP 2026 Dataset\mvtec_anomaly_detection_USED_v5\wood\FAIL\liquid\009.png";
imageChoice = imread(I);

%Run image classification for part type (needed to access pass/fail
% networks)

```

```

[partLabel, partConfidence] = classifyImg(imageChoice,
InspectionSystem.PartClassifier);

%Run image classification function to return label and confidence score for
% PASS/FAIL
[imgLabel, imgConfidence] = classifyImg(imageChoice,...
    InspectionSystem.ConditionClassifiers.(string(partLabel)));

%Show image along with AI classification results
figure

imshow(imageChoice)
title("Your chosen image")
fprintf("Final Condition Classifier label: %s\n", string(imgLabel));
fprintf("Condition Classifier Confidence score: %.2f%%\n", imgConfidence * 100);

fprintf("Final Part Classifier label: %s\n", string(partLabel));
fprintf("Part Classifier Confidence score: %.2f%%\n", partConfidence * 100);

if lower(string(imgLabel)) == "fail"
    %Run image classification function to return label and confidence score for
    % defect type
    [defectLabel, defectConfidence] = classifyImg(imageChoice,
InspectionSystem.DefectTypeClassifiers.(string(partLabel)));
    fprintf("Final Defect Classifier label: %s\n", string(defectLabel));
    fprintf("Defect Classifier Confidence score: %.2f%%\n", defectConfidence * 100);

else
    return
end

```

Task 3 Checkpoint: Combine outputs into a hybrid inspection result

Contributing Members: The whole team

How the system functions

The inspectPart function combines the AI classifiers with the classical image-processing functions into one inspection workflow. The function first uses the part classifier to identify what type of object is in the image. Based on that result, it selects the matching condition classifier to determine whether the part is PASS or FAIL. If the part is classified as FAIL, the function then uses the corresponding defect-type classifier to identify the defect and runs the classical inspection function for that part. The classical function creates the red evidence overlay and defect metrics that show where suspicious regions were found. If the part is classified as PASS, no classical inspection function is performed, the original image is returned, and the defect metrics are set to zero.

Difficulties and trade-offs

One challenge was making sure the outputs from the AI classifiers matched the labels expected by the classical inspection functions. Since each part has different defect types and its own inspection function, a switch statement was needed to send the image to the correct function after the AI classifier identified the part type.

A trade-off was deciding to only run the classical defect segmentation when the AI classified an image as FAIL. This makes the program more efficient and avoids creating defect evidence for passing images. However, if the AI incorrectly classifies a defective image as PASS, the classical inspection function will not run, so the program would not produce an evidence overlay that could help reveal the defect.

A limitation of the combined system is that the final result depends on the accuracy of each classifier before it. The classifiers are used in sequence, starting with the part type, followed by the PASS/FAIL condition and defect type. If an earlier classification is incorrect, the program may select the wrong classifier or classical inspection function for the following steps. As a result, an error in one stage can affect the rest of the inspection workflow.

Another limitation of the hybrid system is that the evidence overlay depends on the accuracy of the classical image-processing functions from Task 2. If those functions produce false detections or miss part of a defect, the final overlay will also contain those errors even if the AI classification is correct.

```
%Below the function, insert a photo to test the entire inspection system
% (make sure that the proper functions are within the same directory as
% this live script)

% ONLY RUN THIS SECTION IF YOU TRAINED YOUR OWN NETWORKS
% DO NOT RUN IF YOU DID NOT TRAIN YOUR NETWORKS

%DELIVERABLE
function [conditionLabel, conditionConfidenceScore,...
    partLabel, partConfidence, typeLabel, typeConfidence, evidenceOverlay,...
    evidenceMetrics] = inspectPart(I, inspectionStruc)

    %Classify part type
    [partLabel, partConfidence] = classifyImg(I, inspectionStruc.PartClassifier);

    %Classify condition (pass/fail)
    [conditionLabel, conditionConfidenceScore] = classifyImg(I,
inspectionStruc.ConditionClassifiers.(string(partLabel)));

    %Determine function to run
    if lower(string(conditionLabel)) == "fail"
```

```

    %Classify defect type
    [typeLabel, typeConfidence] = classifyImg(I,
inspectionStruc.DefectTypeClassifiers.(string(partLabel)));

    % Run the matching classical image-processing function.

    switch lower(string(partLabel))

        case "hazelnut"
            [~, evidenceMetrics, ~, ~, evidenceOverlay] = inspectHazelnut(I,
typeLabel);

        case "wood"
            [~, evidenceMetrics, ~, ~, evidenceOverlay] = inspectWood(I,
typeLabel);

        case "bottle"
            [~, evidenceMetrics, ~, ~, evidenceOverlay] = inspectBottle(I);

        case "grid"
            [evidenceMetrics, evidenceOverlay] = inspectGrid(I, typeLabel);

        otherwise
            error("Unsupported part type: %s", string(partLabel));
    end

else

    % No defect evidence is needed for a passing image.
    typeLabel = categorical("NONE");
    typeConfidence = 1;
    evidenceOverlay = I;

    evidenceMetrics.numComponents = 0;
    evidenceMetrics.maxArea = 0;
    evidenceMetrics.areaRatio = 0;

end
end

% Test function on image for correct output

% Import One Test Image:
% --> User select "Image to Inspect" and pick an image from the dataset
% (change your file type to "All Files (*.*)" when your window opens (bottom right
corner of it))
I = "D:\EPP 2026
Dataset\mvtec_anomaly_detection_USED_v5\hazelnut\FAIL\print\010.png";
img = imread(I);

```

```

[aiConditionLabel, aiConditionScore, aiPartLabel, aiPartConfidence, ...
 aiTypeLabel, aiTypeConfidence, evidence, metrics] = inspectPart(img,...
 InspectionSystem);

% Display for finalLabel = PASS:
if lower(string(aiConditionLabel)) == "pass"
    % Display the original passing image.
    figure;
    imshow(evidence);

    title(sprintf("Final Label: %s", string(aiConditionLabel)));
    fprintf("\nInspection result:\n\n");
    fprintf("Final Condition label: %s\n", string(aiConditionLabel));
    fprintf("Confidence score: %.2f%%\n", aiConditionScore * 100);

    fprintf("Final Part label: %s\n", string(aiPartLabel));
    fprintf("Confidence Score: %.2f%%\n", aiPartConfidence * 100);

    fprintf("Final Defect label: %s\n", string(aiTypeLabel));

% Display for finalLabel = FAIL:
else
    % Display the defect evidence overlay.
    figure;
    imshow(evidence);
    title(sprintf("Final Label: %s", string(aiConditionLabel)));

    % Display results only for failed parts.
    fprintf("Final Condition label: %s\n", string(aiConditionLabel));
    fprintf("Confidence Score: %.2f%%\n", aiConditionScore * 100);

    fprintf("Final Part label: %s\n", string(aiPartLabel));
    fprintf("Confidence Score: %.2f%%\n", aiPartConfidence * 100);

    fprintf("Final Defect Type label: %s\n", string(aiTypeLabel));
    fprintf("Confidence Score: %.2f%%\n", aiTypeConfidence * 100);

    fprintf("Number of suspicious regions: %d\n", metrics.numComponents);
    fprintf("Largest suspicious region: %d pixels\n", metrics.maxArea);
    fprintf("Area ratio: %.4f\n", metrics.areaRatio);
end

```

```

%Below the function, insert a photo to test the entire inspection system
% (make sure that the proper functions are within the same directory as
% this live script)

```

```

% ONLY RUN THIS SECTION IF YOU DID NOT TRAIN YOUR OWN NETWORKS

```

```

% DO NOT RUN IF YOU DID TRAIN YOUR NETWORKS

load testingScriptNetworksAndClasses.mat

% Import One Test Image:
% --> User select "Image to Inspect" and pick an image from the dataset
% (change your file type to "All Files (*.*)" when your window opens (bottom right
corner of it))
I = "D:\EPP 2026
Dataset\mvtec_anomaly_detection_USED_v5\hazelnut\FAIL\print\010.png";
img = imread(I);

[aiConditionLabel, aiConditionScore, aiPartLabel, aiPartConfidence, ...
    aiTypeLabel, aiTypeConfidence, evidence, metrics] = inspectPart(img,...
    InspectionSystem);

% Display for finalLabel = PASS:
if lower(string(aiConditionLabel)) == "pass"
    % Display the original passing image.
    figure;
    imshow(evidence);

    title(sprintf("Final Label: %s", string(aiConditionLabel)));
    fprintf("\nInspection result:\n\n");
    fprintf("Final Condition label: %s\n", string(aiConditionLabel));
    fprintf("Confidence score: %.2f%%\n", aiConditionScore * 100);

    fprintf("Final Part label: %s\n", string(aiPartLabel));
    fprintf("Confidence Score: %.2f%%\n", aiPartConfidence * 100);

    fprintf("Final Defect label: %s\n", string(aiTypeLabel));

    % Display for finalLabel = FAIL:
else
    % Display the defect evidence overlay.
    figure;
    imshow(evidence);
    title(sprintf("Final Label: %s", string(aiConditionLabel)));

    % Display results only for failed parts.
    fprintf("Final Condition label: %s\n", string(aiConditionLabel));
    fprintf("Confidence Score: %.2f%%\n", aiConditionScore * 100);

    fprintf("Final Part label: %s\n", string(aiPartLabel));
    fprintf("Confidence Score: %.2f%%\n", aiPartConfidence * 100);

    fprintf("Final Defect Type label: %s\n", string(aiTypeLabel));
    fprintf("Confidence Score: %.2f%%\n", aiTypeConfidence * 100);

```



```
fprintf("Number of suspicious regions: %d\n", metrics.numComponents);  
fprintf("Largest suspicious region: %d pixels\n", metrics.maxArea);  
fprintf("Area ratio: %.4f\n", metrics.areaRatio);  
end
```

Task 4: Evaluate Inspection System Performance

Contributing members: Marc Angarola and Alexis Arroyo

Background

AI Classification systems are evaluated to ensure they perform as they are meant to. Showing the system images that are similar to what it has learned but has never seen is an optimal strategy. Since the networks output labels and scores, data based on testing can be obtained, calculated, and plotted. Metrics used for quantifying and measuring network performance based on a set of images of similar unseen images are confusion matrices.

A confusion matrix is a square matrix, where each column and row is labeled with the labels the classifier has to predict from. Each label corresponds with its diagonal, meaning the same labels line up with one another on their respective row and column. There are two different sides of the confusion matrix. The side with row labels represents the true side, where the labels correspond to the images that were tested, which are the actual labels of the images. The side with column labels is the prediction side, which are the labels the network knows of and then predicts are the labels of the image/data it sees.

There are four different possibilities that are represented on a confusion matrix: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). True positives is when the label of a test image is true, and the network positively guesses it correctly. True negatives is when the label of a test image is false, and the network does not guess the test image. A false positive is when the label of a test image is false, but the network guesses that label. Finally, a false negative is when the label of a test image is true, but the network does not guess that image. Using these metrics, rates of the system can be determined. Specific rates that are commonly calculated based on these. They are:

- Recall,
- Precision
- Accuracy
- False positives
- False negatives

Accuracy determines how often the predictions the network made are correct. Recall determines, out of all its positive prediction cases, how many it actually gets right. Precision determines, out of all the positive predictions a model makes, how often it makes the correct prediction. On a confusion chart, recall and precision can be determined. Taking the true positives for a class on the column label side and dividing them by the total number of FP's, FN's, and TN's will get you the precision values. The same procedure goes for recall, but this time it is specific to the row labels. Accuracy is self-explanatory. False rejects are data that is good but gets flagged as bad, and false accepts are data that is actually bad, but is flagged as good. False rejects on a confusion matrix are FN's, while false accepts are FP's.

Within manufacturing, these types of metrics can also be used to determine yield rates (the ratio of the amount of data that passed through against the amount of data that should've passed through) and defect rates (the amount of defective items that were found against the amount of total items inspected). TP's and FN's are used to find these, respectively.

The Testing Script (runInspectionSuite)

A testing script was written to be able to train and test the entire inspection system so far. It consists of the following:

- Splitting the original dataset (from Task 1) into a train, validation, and a test set
- Looping through the master function over every image in the test set
- Displaying confusion charts (similar to matrices, but are more visually appealing) for each network
- Calculating the accuracy, recall, precision, false accepts, and false rejects
- Calculating yield rates, defect rates, false accept, and false reject rates based on the condition classifiers matrix
- Display graphs that compare the false rejects and accepts to the accuracy of the condition classifier confusion matrix
- Displaying a montage of true and overlaid images that are deemed to be "failures" of the system

Here, there are 2 different testing scripts.: runInspectionSuiteNOTRAIN.m and runInspectionSuiteTRAIN.m

The difference between these two is simple: One script will take the original data set, split it, train the classifier networks, and test the networks accordingly. The other script does no training at all. Instead, a structure containing all of the networks, along with the testing set that was allocated during its split, can be loaded into the workspace to skip the training process. Each script has a descriptive name that describes what each one does. runInspectionSuiteTRAIN.m utilizes the function "classifyTestSuite" to split the dataset, train, and return the structured full of networks for each classifier,

After training (or no training), the script will begin to establish a separate table to allocate space for storing the filenames of the test set (like in Task 1), and columns that will be used to store info related to testing and evaluating the networks. This table also includes:

- True labels (for each classifier)
- Predicted labels (for each classifier)
- Confidence scores (for each classifier)
- Metrics from evidence overlay
- Logical labels regarding if an image is considered a failure for all classifiers (used in montage)
- Reason statement locators for failure cases (tells the user during the montage the reason for failure)
- The reason statement itself

After the table is established, the master function `inspectPart` will loop through all the images in the test, storing the corresponding information in the newly allocated table. Once all images are classified, confusion matrices for each classifier and their networks are established and saved for later use in calculations.

Confusion charts are established and displayed for each network, along with a general/overall one for each classifier, showcasing the performance of each when classifying the test set. Using the info from the confusion charts of each network, the recall, accuracy, and precision are manually calculated, though they are displayed in each confusion matrix. The recall is the separate row vector at the bottom, while the precision is the separate column vector on the right. The yield and defect rates are calculated and displayed in a table and on a bar graph. The same goes for the false rejects and false accepts, but they are also displayed to compare it with the accuracy since false rejects and accuracy are opposites of one another. Finally, the montage of images that are considered failures is displayed.

These failures that were established to be in the montage are:

- Wrong labels
- Low confidence scores

This applies to all classifiers. Confidence score failures vary. The Part Classifier is considered a confidence failure if the network produces a score of below 95% for any image. The confidence classifier is considered a confidence failure if it produces a score below 88% for any image. Finally, the defect classifier is considered a failure for confidence if it produces a score below 80%. This was determined based upon the amount of images that were available to each classifier during training.

The montage displays not only images, but also reasons for failures. Above the images, a reason will be given as to why the classifier failed. This pertains to maybe an incorrect label per classifier type, as well as low confidence in a classification. If it is both, both reason statements for label and confidence are displayed.

Along with the reason, the true and predicted labels for the images are displayed, along with their confidence score. This supports the reason statements. By default, the condition classifier labels and confidence scores are displayed in the montage. This is dynamically updated as the reason statements change. Label failures take precedence over confidence failures. So, as the statements are displayed, the true and predicted labels as well as

the confidence score are changed to match the classifier that was deemed responsible for the failure. This helps provide proof for the failure reasons outputted.

Results from pre-trained network

To give some insight into the already trained networks for all the classifiers that are a part of the workspace (testingScriptNetworksAndClasses.mat), their evaluation results will be discussed here. It is recommended to run the correct section of code below to see the displays of the evaluation script and follow along.

Now, it can be seen that the number of images across all the defect types is fairly small. This was due to an issue that will be discussed later when it came to partitioning the data for training and testing.

If interested, the metrics for each training session of this network can be found in the repository under "trained_network_structure" and within the subfolder "trained_network_statistics"

The part classifier performed the best, and this can be supported by the training metrics for it that can be found in the repository. The amount of images it had at its disposal was plentiful, allowing for the network to extract all the key features for each part and truly put ResNet18 to the test. Only TP's across the board. I do not believe overfitting occurred due to the amount of images it had to work with, as well as the metrics in the repository to support.

The condition classifier performed the second best, with only 2 FP's. The metrics for training can support this, though they are noticeably noisier across different networks. These results likely occurred due to the training sessions taking longer, with some minor changes in its options to give each network the time to move through the dataset and work on a good balance of training and validation with the smaller amount of images it had to work with.

The same idea applies to the defect classifier when it came to the training time, though the model had been trained for way longer and had its settings adjusted drastically in order to produce decent training metrics, but just good enough to not overfit. They are not a good representation of this network based on the number of images it had to work with, but the team had to make do with what they were given. The weakest part that produced the most negative results was the grid. Across multiple training sessions, the networks always have a difficult time learning the grid across both the condition classifier and the defect classifier. It's graph is fairly noisy, so it is constantly adjusting its weights but still misclassifies it. The main culprit may be the images that are in grid. Some of the defect types blend in with the grid, making it difficult for even our team to determine if the image is a fail or a pass. If there were more images to test with more defining and less hidden defects, perhaps there would be results that would be more proficient.

The defect and yield rates between prediction and actual calculations match what the confusion matrices display, since there were a lot of TP's across the general confusion matrices. There were no false rejects at all, with only around 7% in false accepts, yielding about a 98.7% accuracy. This 7% is due to the FP's produced along the condition classifier.

The montage of images shares quite a few images that are considered failures. As can be seen, a majority of the images deemed as failures are grid. There are many defect issues regarding label and confidence, with some notable aftermarket issues. If a defect label is wrong, the image is never overlaid with a mask, which tells us that this defect type is difficult for the classifier to detect, and some adjustments with options and possible layers may need to be made in order to fix it. But because of the sheer amount of grid images, the grid is a difficult object for the defect classifier to learn. Again, the grid images have some defects in which they blend in and could look like pass images. Regardless, the information here is useful in specifying which classifiers lack for different objects, and also which ones thrive

Struggles

As mentioned before, partitioning the data here for trying to train and then evaluate was a major struggle. Within `classifyTestSuite`, the function `cvpartition` lost its functionality as the test set got introduced, which wasn't something that occurred in task 3 for a simple run of a model to show off the training. The function could not give at least 2 images to each set, for each defect type, of every object when it was split into validation. The split turned from .8 to .2 for just validation and training, to .7 for training, and .15 for the validation and test set (.3 in total). Because of how small the original dataset already was, the function struggled to stratify the allocation and always would forget a defect label for the validation set. This made training impossible for the defect classifier since the number of labels for training must match that of validation, but it didn't. `splitLabels` wouldn't work because the dataset originally was a table, and the function doesn't support splitting tables. Thus, less simple functions had to be used to guarantee at least two images were present in both the testing and validation sets. This is noticeable across the confusion charts since a lot of the defect type per object networks have only 2 TP's, meaning they were only tested on 2 images since that was the only amount that was able to be given to them because of how small the dataset was. This was definitely something that was holding our system back from its true potential.

There was also a struggle with trying to eliminate a NONE defect label from the confusion matrices based on only needing to get the predicted labels to match with the true labels. NONE was part of the labels for the predicted labels, while it was no inclusive in the actual labels for the test set. Lots of time was lost trying to rewrite code for removing certain categories during the defect classifiers displaying of the confusion matrix, believing there was a problem there. Turns out it was in the master function, where the label NONE was being assigned to make the code work in there for outputting. But in reality, in `classifyTestSuite`, the output could be omitted from being output to solve the problem.

NOTE: these may take some time to run depending on the device it is running on. Montage image outputs affect the speed of the script, so if there are a lot, it may take some time to load

```
%THIS SECTION IS USED IF THE USER WISHES TO NOT TRAIN THE NETWORK ON THEIR OWN AND  
% USE PREVIOUSLY STORED DATA ONLY
```

```
%Load testing set from a previous run of "runInspectionSuiteTRAIN.m" and  
% the structure of networks that is associated with it
```

```
load testSetoftestingScriptNetworksAndClasses.mat  
load testingScriptNetworksAndClasses.mat
```

```
%Run the script that will change the directory to match a users directory for the  
dataset (it will  
% open window, directing you to select the latest version of the  
% "mvtec_anomaly_detection_USED" dataset  
run changeDirectoryForTestSet.m  
%}  
%Run inspection script (DOES NOT TRAIN A MODEL. It uses the already trained  
% classifiers along with its corresponding test set  
disp("Running PRETRAINED test script...")  
disp(" ")  
run runInspectionSuiteNOTRAIN.m
```

```
%THIS SECTION IS USED IF THE USER WISHES TO TRAIN THE NETWORK ON THEIR OWN  
% AND USE THEIR OWN DATA (warning, training can take over an hour on lower  
% end devices (possibly 2 hours))
```

```
%Run inspection script (DOES NOT TRAIN A MODEL. It uses the already trained  
% classifiers along with its corresponding test set  
disp("Running TRAINING test script...")  
disp(" ")  
run runInspectionSuiteTRAIN.m
```

Task 5: Test and Evaluate System Robustness

This members that contributed to this section: Marc Angarola and Alexis Arroyo

Background

No background is really needed here, as this section is the same as Task 4, other than the fact that the images of the test set are distorted to varying levels of blur, noise, contrast, and brightness using different MATLAB functions. This tests how robust the system is. Whether or not it can work under variances to images that do not take away any of the key features, networks should be able to recognize and still classify them properly.

Ensure to run the section that utilizes

Results from pre-trained networks

For the part classifier, the results differ quite a bit as there are not only just true positives anymore, but multiple FN's, TN's, and FP's. Recall rates dropped drastically across the objects, as well as their precision. Though the metrics for the parts classifier looked proficient during its training, they cannot seem to put the correct label as to where they used to quite as well before. This may be an augmentation issue, where not enough images were augmented and displayed properly to the network during training

The condition classifier had great recall and precision for TP's, but suffered on FP's, with many FN's, and TN's. Again, the classifiers seem to not be augmented well enough, and now for this classifier and likely the defect classifier, the amount of images it had during training may have cause the system to lose out on robustness.

The defect classifier is classifying labels that don't even belong to its class of labels for the networks, showing an ultimate failure across the board for robustness for likely the same reasons as the condition classifier. This may also prove that based on the training time for the defect classifiers and other options selected for training, this classifier may have overfitted.

The defect and yield rates are close in difference across the prediction and actual rates. Likely this is a result of the better classifier dominating the statistics

The montage of images is extremely large this time, with many having multiple incorrect labels and a lot of mistaken overlays, which is different than the case with no simulated variances in the images. Before, it was mainly just one object; now there are many differing ones.

NOTE: these sections make time to run as they are reevaluating the test set and applying variances

```
%THIS SECTION IS USED IF THE USER WISHES TO NOT TRAIN THE NETWORK ON THEIR OWN AND  
% USE PREVIOUSLY STORED DATA ONLY  
  
%Load testing set from a previous run of "runInspectionSuiteTRAIN.m" and  
% the structure of networks that is associated with it  
  
load testSetoftestingScriptNetworksAndClasses.mat  
load testingScriptNetworksAndClasses.mat  
  
%Run the script that will change the directory to match a users directory for the  
dataset (it will  
% open window, directing you to select the latest version of the  
% "mvtec_anomaly_detection_USED" dataset  
run changeDirectoryForTestSet.m  
%}
```

```
%Run inspection script for robustness test (DOES NOT TRAIN A MODEL. It uses the
already trained
% classifiers along with its corresponding test set
disp("Running PRETRAINED robustness test script...")
disp(" ")
run runInspectionSuiteNOTRAIN_ROBOTEST.m
```

```
%THIS SECTION IS USED IF THE USER WISHES TRAIN THE NETWORK ON THEIR OWN AND
% WITH THE ROBUSTNESS TEST SCRIPT (warning, this script could take a very
% long time to finish running (hours possibly if you are on a lower tier device)
```

```
%Run inspection script for robustness test (DOES NOT TRAIN A MODEL. It uses the
already trained
% classifiers along with its corresponding test set
disp("Running TRAINING robustness test script...")
disp(" ")
run runInspectionSuiteTRAIN_ROBOTEST.m
```

System conclusion/summary

Member contribution for this section: Marc Angarola

The classifier system is the backbone of this system. Without a reliable source, nobody can move forward. Though the classifier was the main culprit for a lot of failures, this system was really limited by the dataset, as there were not a lot of images for the classifiers (except for the part classifier) to be able to be utilized. This could have been solved if the dataset was larger with many more images per defect type of every object, as well as having more defect types amongst objects that were common. Many in the datasets the team went through deemed they were lacking in commonality of defects, which led the team to make the choice of removing things from it. However, this can put more of a workload on the image processing section, as more functions may be needed to account for certain defect types that are not common among different objects.

Because of the dataset with the number of images it had, the team should have probably taken a different approach when setting up the way they wanted to solve the problem. The defect types do not have enough images for a classifier to truly learn and classify the images based on their features, and thus, be the subsystem that carries the entire system as the team planned. Perhaps only having a part classifier and condition (pass/fail classifier) and having the image processing team create algorithms per object rather than defect type. Also, maybe certain functions shouldn't have been used to train the networks, such as `imagePretrainedNetwork`. Perhaps using other functions that allowed more access to change the weights, layers, and more advanced options for training may have proved useful. There were many trade-offs that came up when it came to the methodology of this, and the team fell short with their solution process, and it shows in the testing for robustness across the defect types. Also, knowledge about the MATLAB language proved to be an issue for all members, as

much time was wasted trying to find syntax to implement the system and also correcting errors that would occur as well.

Interpretation of Results

Summarize your findings by interpreting their physical/engineering meaning:

Defect Identification using geometry: Using geometry to detect defects was very useful. Take the grid for example: Any region who's binarized hole is abnormally small or large, is most likely a defect. An abnormally small area could be caused by glue on top of the grid, or metal contamination around the hole, similarly, a bigger sized hole could be caused by the grid being broken, and allowing two holes to merge into one object.

Issues with the mean: Using the mean for a large set of images was not good. This is because of multiple outliers being able to skew the mean heavily. (For example, an image with 5 normal holes, and 3 very small holes will cause the mean to be skewed downwards, pulling it away from what a true physical baseline of a non defective hole should be).

A solution against the mean: Instead of using the unreliable mean, we can use the median, and the median absolute deviation (a summary and a brief example of it was written under task 3 of grid). `mad()` works well because it takes the standard hole size, and measures the variance around that standard hole size without being influenced by small or large artifacts (such as very small circles, or extra noise that went by unnoticed).

What are some limitations of your work?

High frequency geometry: A clear limitation (or maybe a lack of proficiency) was shown to us with the screw object. There was an inability to handle high frequency surface geometry which was caused by the screw thread.

Unreliability: Due to the limited dataset, the training for the classifiers become unreliable as their metrics never consistent across different training sessions. This is due to the random splitting of the dataset, and some images in it were more difficult than others for the classifier to learn

Majority vs Minority: Another question we can ask ourselves is what would happen if the object is VERY defected? Meaning, what if instead of one glue, or one break in the grid, there is now 20 pieces of glue, and multiple breaks.

This would cause a huge issue, because most of our calculations (median and mad) is assuming that we are dealing with an image that is GOOD for the majority of it, and bad for a minority of it, but as soon as we flip that, then the calculations don't work, because now they assume the normal is the defect, and the anomaly is the good.

What are practical next steps?

Create safe measures when the baseline becomes a defect.

This should be done BEFORE using a computed baseline such as `median(stats.Area)` or `mad()`.

One way we could check is to test whether the median falls within a physically realistic range derived from manufacturing specifications and documentation.